

LABORATORY EXPERIMENTAL PROTOCOL

Applied Massive Parallelism: CUDA Python, Thread Geometries, and Hardware Latency Pitfalls

Student Name:	Yermurat Ainat	Student ID:	230103161
Session / Date:	09.10.2026	Assigned GPU ID:	_____

CRITICAL LAB POLICIES & AI-IMMUNITY PROTOCOL

- **TIME ALLOCATION:** Exactly 75 minutes. Keep pacing strictly aligned with the time markers.
- **INDEPENDENT WORK POLICY:** This protocol is fully self-directed. Follow each step sequentially.
- **AI-IMMUNITY NOTICE:** Generic code copy-pasted from AI models will FAIL the grading criteria. Grading is evaluated strictly on: (1) your unique hardware telemetry numbers, (2) empirical benchmark values from your allocated physical T4 GPU node, and (3) a cryptographic checksum seeded by your Student ID.
- **DELIVERABLE:** You will save your completed Colab notebook (.ipynb) and write your empirical observations directly into this task sheet.

SECTION 1: ENVIRONMENT PROVISIONING & HARDWARE SETUP (15 MIN)

Follow these exact steps to provision your high-performance GPU cloud runtime. Do NOT skip any step.

[Step 1.1] Open your web browser (Chrome or Edge recommended) and navigate directly to: <https://colab.research.google.com>

[Step 1.2] Sign in with your university Google account. In the welcome dialog, click 'New notebook' (or click File → New notebook).

[Step 1.3] In the top-left title bar, change the notebook name from 'Untitled0.ipynb' to: CUDA_Lab01_<YourStudentID>.ipynb

[Step 1.4] Enable Hardware Acceleration: On the top menu bar, click: Runtime → Change runtime type.

[Step 1.5] In the modal popup, locate 'Hardware accelerator'. Click the dropdown and select 'T4 GPU' (under standard runtime).

[Step 1.6] Click the blue 'Save' button. Wait 5-10 seconds while the cloud instance provisions.

[Step 1.7] Check connection status: Look at the top-right corner of the screen. You should see a green checkmark next to 'Connected (T4)' showing RAM and Disk gauges. If it says 'Connect', click it.

Task 1.1: Hardware Telemetry Extraction (AI-Immune Verification)

In the very first code cell of your notebook, paste and execute the following command (Shift + Enter):

```
!nvidia-smi
```

In a second code cell, execute the following Python hardware inspection script to query the Numba CUDA driver interface directly:

```
from numba import cuda
import os

device = cuda.get_current_device()
print(f"Device Name: {device.name.decode('utf-8')}")
print(f"Compute Capability: {device.compute_capability}")
print(f"Multiprocessor Count (SMs): {getattr(device, 'MULTIPROCESSOR_COUNT', 'N/A')}")
print(f"Max Threads Per Block: {device.MAX_THREADS_PER_BLOCK}")
print(f"Max Block Dimensions: {device.MAX_BLOCK_DIM_X}, {device.MAX_BLOCK_DIM_Y},
```

```
{device.MAX_BLOCK_DIM_Z}")
print(f"Warp Size: {device.WARP_SIZE}")
```

MANDATORY SUBMISSION FIELD: Fill in the exact telemetry values printed by your instance below:

NVIDIA Driver Version (from nvidia-smi table)	[Record: 580.82.07 _____]
CUDA Version Supported by Driver	[Record: 13.0 _____]
Allocated GPU Name (e.g., Tesla T4)	[Record: Tesla T4 _____]
Total Available VRAM (e.g., 15360 MiB)	[Record: 15360MiB _____]
Compute Capability Major.Minor (e.g., 7.5)	[Record: (major=7, minor=5) _____]
Max Threads Per Block (hardware limit)	[Record: 1024 _____]

SECTION 2: THE LIE OF GPU SPEED & THE CROSSOVER EXPERIMENT (25 MIN)

Novice programmers assume: 'If I put my code on the GPU, it will instantly run 100x faster.' This is false. A GPU is connected to the CPU via the PCIe bus—a narrow physical copper bus with high latency. If the data is small, the overhead of copying bytes to the GPU dwarfs the computation time. In this experiment, you will prove this empirically.

Task 2.1: Implement the Vector Doubling Kernels

Add a new code cell and paste the following baseline comparison framework:

```
import numpy as np
import time
from numba import cuda

# 1. CPU Reference Implementation (Pure Python loop)
def double_cpu_pure(arr):
    out = np.empty_like(arr)
    for i in range(len(arr)):
        out[i] = arr[i] * 2.0
    return out

# 2. GPU Kernel Implementation
@cuda.jit
def double_gpu_kernel(d_in, d_out):
    # Calculate unique 1D thread ID
    idx = cuda.grid(1)
    if idx < d_in.shape[0]:
        d_out[idx] = d_in[idx] * 2.0
```

Task 2.2: Benchmark Across Orders of Magnitude

Execute the benchmark harness across different array sizes N from 100 up to 10,000,000. For each N, measure two distinct GPU metrics:

- Total Roundtrip GPU Time:** Host-to-Device copy (PCIe) + Kernel Execution + Device-to-Host copy (PCIe).

2. **Kernel-Only Compute Time:** Data is already pre-loaded into VRAM; measure ONLY the GPU execution time.

```
def benchmark_run(N, student_id_seed=1234):
    np.random.seed(student_id_seed)
    h_in = np.random.rand(N).astype(np.float32)

    # Measure CPU Pure Loop (skip for N >= 10M if too slow) if N <=
    1_000_000:
    t0 = time.perf_counter()
    _ = double_cpu_pure(h_in)
    cpu_time_ms = (time.perf_counter() - t0) * 1000.0
    else:
    cpu_time_ms = float('nan') # Extrapolate or skip

    # GPU Setup threads_per_block =
    256
    blocks_per_grid = (N + threads_per_block - 1) // threads_per_block

    # Warmup compilation run
    d_warmup = cuda.to_device(np.ones(10, dtype=np.float32))
    double_gpu_kernel[1, 32](d_warmup, d_warmup)
    cuda.synchronize()

    # GPU Roundtrip Measurement (Includes PCIe transfers) t0 =
    time.perf_counter()
    d_in = cuda.to_device(h_in)
    d_out = cuda.device_array_like(d_in) double_gpu_kernel[blocks_per_grid,
        threads_per_block](d_in, d_out) h_out = d_out.copy_to_host()
    cuda.synchronize()
    gpu_total_ms = (time.perf_counter() - t0) * 1000.0

    # GPU Compute-Only Measurement (Pre-allocated in VRAM) t0 =
    time.perf_counter()
    double_gpu_kernel[blocks_per_grid, threads_per_block](d_in, d_out) cuda.synchronize()
    gpu_compute_ms = (time.perf_counter() - t0) * 1000.0

    return cpu_time_ms, gpu_total_ms, gpu_compute_ms
```

MANDATORY EXPERIMENTAL DATA TABLE: Record your empirical times (in milliseconds) below:

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner (CPU vs GPU)
100	0.045	1.767	0.130	CPU
1,000	0.280	0.749	0.128	CPU
10,000	3.274	0.449	0.061	GPU
100,000	26.770	0.627	0.068	GPU
1,000,000	270.068	4.725	0.113	GPU
5,000,000	nan	15.797	0.311	GPU
10,000,000	nan	25.108	0.452	GPU

Analysis Questions (Complete in ink or typed):

Q1. What is the empirical Crossover Point N^* on your machine? (i.e., at what array size did GPU Total Time finally become faster than CPU Time?):

Answer: The empirical crossover point is **$N = 10,000$** . At this array size, the GPU Total Time (0.449 ms) becomes faster than the CPU Time (3.274 ms). At $N = 1,000$, the GPU Total Time (0.749 ms) is still slower than the CPU Time (0.280 ms). Therefore, $N = 10,000$ is the first tested array size at which the GPU outperforms the CPU in total execution time.

Q2. At $N = 100$, why is GPU Total Time dramatically slower than CPU Time? Compare the 'GPU Total' column against the 'GPU Kernel Only' column to explain where 95%+ of the elapsed time was actually spent:

Answer: : At $N = 100$, the GPU Total Time is 1.767 ms, while the GPU Kernel Only Time is just 0.130 ms. Therefore, approximately 92.6% of the total GPU time is spent on overhead rather than kernel execution. This overhead includes transferring data between CPU and GPU memory and launching the CUDA kernel. The actual computation takes only a small fraction of the total time. As a result, the CPU is faster for very small arrays because it avoids GPU data-transfer and kernel-launch overhead.

SECTION 3: THREAD GEOMETRY & BOUNDARY MEMORY CORRUPTION (25 MIN)

GPU execution hardware does not operate in arbitrary numbers; it groups threads into rigid blocks (standard sizes: 128, 256, 512, 1024). When your data size N is not an exact multiple of the block size, you **MUST** over-provision threads and enforce an explicit boundary guard. In this task, you will intentionally trigger memory corruption by breaking this rule.

Task 3.1: The 'Missing Guard' Disaster Experiment

In this task, we set array size $N = 1,000$ and launch blocks of 256 threads. To cover 1,000 elements, how many blocks must be launched? Ceiling division: $\text{ceil}(1000 / 256) = 4$ blocks = 1,024 threads. Look at threads 1000 through 1023: they have no valid array element

Write down what exact error message or console behavior was displayed on your screen:

Observed behavior / Console output from defective kernel:

[Write here: The defective kernel executed without displaying an error. The output showed that 1000 elements were processed, and the last element was 999.0. Although 1024 threads were launched for an array of 1000 elements, the kernel had no boundary check. Therefore, 24 threads attempted to access elements outside the array bounds. In this run, the invalid memory accesses did not produce a visible error, but they could cause memory corruption or a CUDA error.]

Task 3.2: Spatial Thread Coordinate Inspector (N = 37)

To prove how threads are spatially addressed, write a diagnostic kernel for an odd data size: N = 37 with a block size of 8 threads. Calculate the grid dimensions and log the exact thread coordinates.

```
@cuda.jit
def inspect_threads_kernel(log_block, log_thread, log_global, log_active, N):
    t_idx = cuda.threadIdx.x
    b_idx = cuda.blockIdx.x
    g_idx = cuda.grid(1)

    if g_idx < log_block.shape[0]:
        log_block[g_idx] = b_idx
        log_thread[g_idx] = t_idx
        log_global[g_idx] = g_idx
        log_active[g_idx] = 1 if g_idx < N else 0

# Configuration
N = 37
threads_per_block = 8
blocks_per_grid = (N + threads_per_block - 1) // threads_per_block # Exactly 5 blocks = 40 threads

log_b = cuda.device_array(blocks_per_grid * threads_per_block, dtype=np.int32)
log_t = cuda.device_array(blocks_per_grid * threads_per_block, dtype=np.int32)
log_g = cuda.device_array(blocks_per_grid * threads_per_block, dtype=np.int32)
log_a = cuda.device_array(blocks_per_grid * threads_per_block, dtype=np.int32)

inspect_threads_kernel[blocks_per_grid, threads_per_block](log_b, log_t, log_g, log_a, N)
cuda.synchronize()

print(f"{'Global ID':<10} | {'Block ID':<10} | {'Thread ID':<10} | {'Status':<10}")
print("-" * 50)
for i in range(32, 40): # Print the critical boundary transition (Block 4) status
    = "ACTIVE" if log_a[i] == 1 else "IDLE (GUARDED)"
    print(f"{log_g[i]:<10} | {log_b[i]:<10} | {log_t[i]:<10} | {status:<10}")
```

MANDATORY CALCULATION: For N = 37 and Block Size = 8:

- Total Blocks Allocated: [5] • Total Hardware Threads Launched: [40]
- Number of Active Working Threads: [37] • Number of Idle Guarded Threads: [3]

SECTION 4: THE RACE CONDITION CRIME SCENE (PARALLEL REDUCTION) (25 MIN)

A race condition occurs when multiple threads attempt to read and write to the same memory address concurrently without synchronization. In this task, you will witness a massive data corruption event firsthand by attempting to sum an array in parallel.

Task 4.1: The Naive Reduction Bug (Data Corruption)

In this experiment, 10,000 threads simultaneously attempt to add 1.0 to a single scalar variable total[0]. In sequential code, 10,000 x 1.0 = 10,000. What happens in parallel?

```
# BUGGY NAIVE SUMMATION
@cuda.jit
def naive_sum_kernel(d_arr, d_total):
    idx = cuda.grid(1)
if idx < d_arr.shape[0]:
# DATA RACE: Thousands of threads execute Read-Modify-Write at the exact same clock
cycle!
d_total[0] += d_arr[idx]

# CORRECT ATOMIC SUMMATION
@cuda.jit
def atomic_sum_kernel(d_arr, d_total):
    idx = cuda.grid(1)
if idx < d_arr.shape[0]:
# HARDWARE ATOMIC LOCK: Hardware serializes updates to prevent collisions cuda.atomic.add(d_total, 0,
d_arr[idx])
```

MANDATORY EXPERIMENTAL PROOF: Run naive_sum_kernel across 5 consecutive trials on an array of 50,000 ones (Expected Sum = 50,000.0). Record the actual calculated values below:

Trial #	Target Sum	Naive Kernel Output	Deviation / Lost Updates
Trial 1	50,000.0	2	49998
Trial 2	50,000.0	2	49998
Trial 3	50,000.0	2	49998
Trial 4	50,000.0	2	49998
Trial 5	50,000.0	2	49998

Now run atomic_sum_kernel on the same 50,000 elements:

- Atomic Sum Output: [50000.] (Does it equal exactly 50,000.0? Yes / No) YES
- Performance Penalty Comparison: Measure runtime of atomic_sum vs naive_sum:
 - Naive Kernel Time: [12.274 ms (average)] vs. Atomic Kernel Time: [29.248 ms (average)ms]
 - Explain why atomic operations incur a massive execution penalty:

Atomic operations incur a performance penalty because all GPU threads attempt to update the same memory location. These updates must be performed safely, so competing operations may be serialized. This contention reduces parallelism and increases execution time. In my experiment, the atomic kernel took an average of 29.248 ms, compared with 12.274 ms for the naive kernel, making it approximately 2.38 times slower. However, the atomic kernel produced the correct sum of 50,000, while the naive kernel produced an incorrect sum of 2 due to race conditions and lost updates.

SECTION 5: 2D GRID IMAGE/MATRIX PROCESSING (20 MIN)

Real-world graphics, computer vision, and deep learning operate on 2D and 3D data grids. In CUDA, threads and blocks can be configured in 2D using (x, y) coordinates. You will compute a 1024 x 1024 Euclidean radial vignette filter.

```
import math

@cuda.jit
def radial_vignette_2d(d_canvas, width, height, center_x, center_y): #
    # Retrieve 2D spatial coordinates directly from the grid
```

```

dx = float(x - center_x) dy =
    float(y - center_y)
dist = math.sqrt(dx * dx + dy * dy)
max_dist = math.sqrt(float(center_x**2 + center_y**2)) intensity = 1.0 -
    (dist / max_dist)
    if intensity < 0.0:
        intensity = 0.0
d_canvas[y, x] = intensity

# 2D EXECUTION CONFIGURATION
WIDTH, HEIGHT = 1024, 1024
canvas = np.zeros((HEIGHT, WIDTH), dtype=np.float32)
d_canvas = cuda.to_device(canvas)

# 2D Blocks (16x16 = 256 threads per block) threads_2d
= (16, 16)
blocks_2d = (
    (WIDTH + threads_2d[0] - 1) // threads_2d[0], (HEIGHT +
        threads_2d[1] - 1) // threads_2d[1]
    )

radial_vignette_2d[blocks_2d, threads_2d](d_canvas, WIDTH, HEIGHT, WIDTH // 2, HEIGHT // 2)
    cuda.synchronize()
result_canvas = d_canvas.copy_to_host()
print(f"Generated 2D matrix shape: {result_canvas.shape}. Center value: {result_canvas[512,
    512]:.3f}")

```

MANDATORY 2D GEOMETRY VERIFICATION

- **Grid Dimensions:** Blocks in X: 64 | Blocks in Y: 64 | Total Blocks: 4096
- **Verified Pixel Value at Center Point (512, 512):** 1.0
- **Verified Pixel Value at Outer Corner Point (0, 0):** 0.0

SECTION 6: FINAL AI-IMMUNE VERIFICATION CHECKSUM (10 MIN)

To verify that all kernels were physically executed on your allocated GPU and prevent plagiarism, run the official validation script below in your final Colab cell. It hashes your student ID, hardware node properties, and kernel execution buffers into a tamper-evident 16-character token.

```

import hashlib

def generate_lab_checksum(student_id_str, d_canvas, exp_table_crossover): h
    = hashlib.sha256()
h.update(student_id_str.strip().encode('utf-8')) #
    Include GPU name
    h.update(cuda.get_current_device().name)
# Include sample of compute output
    h.update(d_canvas.copy_to_host()[:10, :10].tobytes())
    h.update(str(exp_table_crossover).encode('utf-8'))
    digest = h.hexdigest()[:16].upper()
print("=" * 60)
print(f"OFFICIAL VERIFICATION CHECKSUM TOKEN: {digest}")
print("=" * 60) return
    digest

# REPLACE WITH YOUR ACTUAL STUDENT ID AND EMPIRICAL CROSSOVER N* VALUE
STUDENT_ID = "YOUR_STUDENT_ID_HERE"
CROSSOVER_N = 100000 # Example: your value from Section 2
generate_lab_checksum(STUDENT_ID, d_canvas, CROSSOVER_N)

```

53ABFF0A016F45ED

53ABFF0A016F45ED

FINAL SUBMISSION TOKEN & INTEGRITY CHECK

PASTE YOUR UNIQUE 16-CHARACTER VERIFICATION TOKEN HERE:

53ABFF0A016F45ED

FINAL SUBMISSION CHECKLIST:

- ☐ Notebook saved as CUDA_Lab01_<StudentID>.ipynb in Google Drive
- ☐ All code cells executed sequentially with output clear and visible (no cleared outputs)
- ☐ Exported as .ipynb (File → Download → Download .ipynb) and uploaded to course portal
- ☐ This task sheet filled out completely and submitted to the instructor